

OASIS: Feedback-Driven Statistics Correction for Cost-Based Query Optimizers

Author Names Redacted for Review

Affiliation Redacted
redacted@review.org

Abstract. In large-scale analytical databases, the accuracy of cost-based optimizers (CBOs) depends critically on the freshness of column-level statistics. In practice, column statistics are refreshed only by expensive full-table **ANALYZE** passes scheduled daily or less frequently, while tables undergo continuous DML workloads—creating a persistent *staleness gap* that causes systematic selectivity estimation errors, suboptimal join orders, and measurable query slowdowns.

We present **OASIS** (**O**nline **A**daptive **S**tatistics **I**nference **S**ystem), a feedback-driven framework for *statistics correction* at the column level. Unlike plan-level feedback methods that adjust optimizer hints or join orders, OASIS operates directly on the statistics representation consumed by the CBO. It repairs stale *single-column* histograms from query execution feedback already produced during predicate evaluation, without re-scanning tables or modifying the optimizer, thereby improving the accuracy of optimizer inputs that rely on single-column statistics. Because different DBMSs consume such statistics differently, the downstream impact on final plan quality is engine-dependent; OASIS is therefore complementary to techniques that model multi-column correlations or intervene at the plan level. OASIS encodes each correction instance as a compact feature tensor and predicts corrected quantiles with a lightweight *attention-pooled MLP* trained on simulation-derived drift scenarios. To make statistics correction portable across DBMSs, OASIS further introduces *statistics-format conversion interfaces* that map engine-specific statistics layouts to a canonical full-distribution view and back, covering both self-contained histograms and tightly coupled formats such as MCV+histogram designs.

Experiments across synthetic drift scenarios confirm that OASIS reduces Q-Error by up to 62.0% over stale priors while remaining lightweight at deployment time. Trained once on diverse drift scenarios, the same 38K-parameter checkpoint deploys zero-shot across heterogeneous columns and schemas. Across unseen initial distributions at $q=10$, OASIS still improves over the stale prior by 44.4%–60.8%. In end-to-end TPC-DS evaluation, OASIS recovers 52.0% and 75.7% of the execution-time improvement achieved by Full ANALYZE over stale statistics on PostgreSQL 14 and MySQL 8.0.31+, respectively.

Keywords: Statistics correction · Cardinality estimation · Histogram calibration · Cost-based optimizer · Query feedback · Online learning

1 Introduction

Modern analytical query engines rely on column-level histograms to estimate predicate selectivities, which drive join-order selection and operator-algorithm choices in cost-based optimizers (CBOs). Column statistics are refreshed by periodic **ANALYZE** passes—equivalent to full table scans—that are operationally expensive and typically scheduled daily or less frequently. Meanwhile, tables undergo continuous DML workloads (inserts, deletes, updates), causing a persistent *staleness gap*: the CBO’s selectivity estimates $\hat{s}$ diverge from true selectivities s^* , leading to suboptimal plans. For example, underestimating a base table’s cardinality can cause the optimizer to select a Nested-Loop Join that runs magnitudes slower than a Hash Join chosen after correction.

Existing approaches address this problem only partially. Frequent **ANALYZE** reruns are operationally prohibitive for large tables and disrupt production query traffic. Learned cardinality estimators such as NeuroCard [24] and FACE [25] bypass histograms entirely, but they produce point estimates rather than corrected histograms, making integration with existing CBOs difficult, and require access to raw table data for training. *Plan-level* feedback methods like LEO [21] and Bao [14] adjust optimizer hints or plan choices based on execution history, but do not repair the underlying statistics—the CBO still consumes stale histograms, and estimation errors persist across all queries referencing the affected columns. In contrast, *statistics-level* correction—the focus of this work—repairs the histogram representation itself, allowing the unmodified CBO and its entire cost model to benefit from improved input. Self-tuning histograms [1] operate at the statistics level but require periodic data scans to validate adjustments, which may be unavailable when statistics are maintained separately from the data store (e.g., external metadata catalogs).

We observe that CBOs continuously generate useful implicit feedback during normal query execution: the gap between estimated selectivity $\hat{s}$ and actual selectivity s^* is directly observable from operator-level runtime statistics. OASIS turns this per-predicate, per-column signal into online statistics correction before the histogram reaches the CBO, without rereading raw table data. Its direct effect is therefore to improve the accuracy of optimizer inputs that depend on stale *single-column* statistics. Whether and how strongly that improvement translates into better plans remains a property of the target DBMS, since each optimizer consumes single-column histograms, MCV lists, NDV summaries, and other statistics differently. In end-to-end evaluation on TPC-DS, OASIS recovers 52.0% and 75.7% of the execution-time improvement achieved by Full **ANALYZE** over stale statistics on PostgreSQL 14 and MySQL 8.0.31+, respectively.

At a high level, OASIS formulates statistics correction as a regression problem over the prior histogram and a recent observation window, trains once on simulation-derived drift scenarios, and deploys the same 38K-parameter checkpoint zero-shot across columns and schemas. Integration requires only two lightweight hooks—a query-completion listener and a statistics-assembly intercept—so OASIS remains intentionally scoped to single-column histogram

staleness. It repairs marginal distributions of individual columns, not joint distributions across attributes, and therefore remains complementary to methods that target multi-column correlations, join-level cardinality errors, or plan-level optimization decisions. In particular, even learned multi-column estimators such as copula-based models still rely on accurate one-dimensional marginals as inputs, making single-column statistics quality a foundational concern rather than an obsolete one.

Our contributions are as follows:

1. **Problem formalization** (§2): We define statistics correction as a regression task over prior histograms and query feedback windows, with Q-Error as the optimization objective.
2. **Feature representation and correction algorithm** (§3): A model-agnostic feature representation encoding the prior histogram and a causal observation window, consumed by an *attention-pooled MLP* trained on diverse drift scenarios, achieving robust performance across drift intensities.
3. **Non-invasive system integration** (§3.4, §3.5): Per-conjunct operator instrumentation and a plugin-based feedback listener, with correction injected at the statistics assembly layer via dependency injection. The design is engine-agnostic, requiring only a feedback listener and a histogram modification hook.
4. **Cross-database portability via statistics-format conversion interfaces** (§3.6): A canonical full-distribution interface with lightweight bidirectional converters for engines that use different degrees of histogram coupling: direct integration for self-contained histograms (MySQL), and format translation for tightly coupled MCV / hybrid summaries (PostgreSQL, Oracle, SQL Server). This makes statistics correction broadly deployable rather than restricted to engines exposing standalone histograms. Validated with exact mass conservation, machine-precision round-trip fidelity, and sub-millisecond overhead on typical PostgreSQL configurations.
5. **Systematic evaluation** (§4): Ablation experiments across drift intensities ($q \in \{1, \dots, 30\}$), sensitivity analysis, and end-to-end TPC-DS studies on PostgreSQL and MySQL confirm up to **62%** Q-Error reduction and recovery of **52.0%** and **75.7%** of Full ANALYZE’s wall-clock improvement, respectively.

2 Background and Problem Formulation

2.1 Column Histograms and the Staleness Gap

A CBO represents the per-column value distribution as an *equi-depth histogram* with B equal-probability buckets, defined by $(B - 1)$ internal boundary values $v_1, \dots, v_{B-1}$ (equivalently, quantile values at levels $p_1, \dots, p_{B-1}$). Selectivity of a range predicate $\sigma = (\text{col} < v)$ is computed as $\hat{s} = \text{CDF}_H(v)$ via piecewise-linear CDF interpolation.

Histograms are refreshed by running **ANALYZE**—a full table scan—which is expensive and therefore scheduled infrequently (typically daily or less). Continuous

DML operations between refreshes cause the stored histogram H_0 to diverge from the true distribution H^* , inducing estimation errors.

Format agnosticism and cross-database portability. Our problem formulation and algorithms operate on normalized equi-depth boundaries and are agnostic to the underlying histogram encoding (§3.2). Whether the engine stores statistics as equi-depth buckets, V-optimal partitions, or streaming sketches (e.g., KLL [9]), OASIS requires only that a canonical quantile / CDF view can be extracted and written back.

This requirement is more subtle than it first appears: in many DBMSs, the “histogram” is not a standalone editable object but is tightly coupled with auxiliary summaries such as MCV lists or density vectors. Consequently, the practical limitation of prior histogram-correction methods is not only accuracy—it is *portability*. OASIS addresses this with a lightweight statistics-format conversion interface (§3.6) that maps engine-specific statistics layouts to a canonical full-distribution representation, applies correction there, and translates the result back. This yields direct deployment for self-contained histograms (MySQL, MariaDB), and safe bidirectional conversion for tightly coupled formats such as PostgreSQL / Oracle (MCV + residual histogram) and SQL Server (density vector + histogram).

2.2 Selectivity Feedback from Query Execution

Upon query completion, the execution engine reports per-operator input and output row counts. For a filter operator evaluating predicate ϕ at value v , we derive the estimated selectivity $\hat{s}$ from the CBO’s row-count estimates and the actual selectivity s^* from the execution statistics. We record each observation as:

$$o_i = (\phi_i, v_i, v_i^{\text{upper}}, \hat{s}_i, s_i^*),$$

where $\phi_i \in \{<, \leq, >, \geq, =, \text{BETWEEN}\}$ is the predicate type, v_i (and optional v_i^{upper} for BETWEEN) the filter value. Importantly, these observations are collected *while the engine is already reading table pages and evaluating filter conditions for normal query execution*; OASIS does not trigger an extra table pass. The collection path therefore adds only bounded bookkeeping and emits at most one observation record per filter conjunct, so each query writes at most as many observation tuples as there are conditions in its predicate.

2.3 Statistics-Level vs. Plan-Level Feedback

Before formalizing our problem, we clarify the distinction between two complementary approaches to leveraging query execution feedback:

Plan-level feedback. Methods such as LEO [21], Neo [15], and Bao [14] learn from execution history to adjust *optimizer hints*, *cardinality hints*, or *plan choices* directly. For example, LEO maintains a feedback loop that modifies the optimizer’s

cost model parameters or overrides specific join-order decisions based on observed plan performance. These methods are effective at improving plan selection for recurring query patterns, but they do not repair the underlying column statistics: the CBO still consumes the stale histogram H_0 , and estimation errors persist across all queries that reference the affected columns.

Statistics-level feedback (this work). OASIS operates at a finer granularity: it corrects the *histogram representation itself* before it reaches the CBO. By repairing $H_0 \rightarrow \hat{H}$ at the statistics layer, the unmodified CBO and its entire cost model—join ordering, operator selection, memory budgeting—benefit from improved input. Corrections propagate across *all* queries that reference the column, not just those matching a learned plan template. Furthermore, statistics-level correction is orthogonal to plan-level methods: a system can deploy both simultaneously, with OASIS providing better input statistics and LEO/Bao refining plan selection on top of those statistics.

2.4 Problem Statement

Let H_0 be the stale prior histogram captured at ANALYZE time and $\mathcal{O} = \{o_1, \dots, o_K\}$ a window of K most-recent query observations for a given column. The *Q-Error* [17] of an estimate $\hat{s}$ for true selectivity s^* is:

$$\text{QErr}(\hat{s}, s^*) = \max\left(\frac{\hat{s}}{s^*}, \frac{s^*}{\hat{s}}\right) \geq 1. \quad (1)$$

Problem 1 (Statistics Correction). Given the stale prior H_0 and an observation window $\mathcal{O}$, produce corrected column statistics $\hat{H}$ such that:

$$\mathbb{E}_\sigma [\text{QErr}(\text{CDF}_{\hat{H}}(\sigma), s_\sigma^*)] < \mathbb{E}_\sigma [\text{QErr}(\text{CDF}_{H_0}(\sigma), s_\sigma^*)],$$

where the expectation is over future predicates σ drawn from the workload distribution.

The solution must satisfy three operational constraints: **(D1)** no table scan (derive $\hat{H}$ from H_0 and $\mathcal{O}$ only); **(D2)** low latency (<200 ms at query-planning time); **(D3)** graceful degradation (when $\mathcal{O}$ is sparse, $\hat{H} \approx H_0$).

Scope. OASIS targets *single-column histogram staleness* for histogram-sensitive predicates. Its goal is to improve the fidelity of the optimizer’s single-column inputs—that is, the marginal distributions consumed when the DBMS estimates predicate selectivities from per-column statistics. OASIS does not attempt to repair multi-column correlations, join-level cardinality interactions, or other estimation errors that are not mediated by a single column’s statistics object. Consequently, it does not model joint distributions across attributes: if a DBMS’s dominant planning errors stem from cross-column dependence, then OASIS can improve only the single-column portion of that error budget. In that sense, OASIS is best viewed as a complement to, rather than a replacement for, techniques that model cross-column dependence, extended statistics, or join output cardinalities.

3 System Design

3.1 System Overview

The OASIS pipeline intercepts stale prior histograms H_0 before they reach the CBO. A *Feedback Collector* extracts per-conjunct observations o_i from completed queries (§3.4); a *Feature Tensorizer* encodes H_0 and $\{o_i\}$ into a fixed-size tensor $\mathbf{x}$ (§3.2); an *Correction Model* maps $\mathbf{x}$ to corrected quantiles $\hat{\mathbf{v}}$; and $\hat{\mathbf{v}}$ is reconstructed into the engine’s native histogram format and passed transparently to the CBO. The entire pipeline touches only two engine integration points (§3.4, §3.5), making OASIS portable across query engines without modifying the CBO or planner.

3.2 Feature Representation

Histogram normalization. The correction pipeline operates on normalized equi-depth boundaries. We extract $(B - 1)$ quantile values $\mathbf{v}$ from H_0 and min-max normalize to $[0, 1]$, yielding $b_0=0$, $b_i=v_i$, $b_B=1$. This decouples the model from physical value ranges and histogram encoding (equi-depth, V-optimal, KLL sketch, etc.).

Tensor layout. The feature tensor $\mathbf{x}$ consists of four blocks: **(1)** Prior block ($B-1$ dims): normalized quantile values from H_0 . **(2)** Meta block (3 dims): null fraction, observation count ratio, bucket count. **(3)** Observation block ($K \times 12$ dims): for each of K observations, a 12-dim encoding (6-dim predicate one-hot + 6 numeric features: v , v^{upper} , $\hat{s}$, s^* , has_upper, span), ordered oldest-to-newest (causal order). **(4)** Mask block (K dims): binary validity indicators. Total dimension: $d = (B-1) + 3 + 12K + K = 220$ with $B=10$, $K=16$. The format is model-agnostic (MLP, LSTM, Transformer compatible).

3.3 OASIS Model: Attention-Pooled MLP

The OASIS correction model combines three lightweight components: a compact encoder for the stale prior histogram, attention pooling over the observation window, and a residual MLP that predicts a correction delta for the quantile vector. Intuitively, the prior encoder captures the coarse shape of the stale distribution, attention identifies which recent observations are most informative, and the residual head updates the prior rather than regenerating the histogram from scratch. This keeps inference CPU-friendly while still capturing non-linear interactions between the prior and feedback history.

Output validity. OASIS predicts only the internal quantile boundaries of a fixed quantile grid; the bucket masses themselves remain those implied by the canonical equi-depth levels. To ensure that the corrected histogram is always a valid CDF summary, we treat the network output as an unconstrained residual and apply a lightweight validity projection after inference: predicted normalized quantiles

are clamped to the legal value range and then forced to be non-decreasing by monotonic projection (implemented in the paper-facing runner as cumulative-max clipping with fixed endpoints). This preserves the original minimum and maximum, prevents negative bucket widths or CDF inversions, and guarantees that the corrected statistics can be translated back into an engine-native histogram object.

Training. The model is trained offline on simulation-derived ground truth from the compound-drift process described in §4. For each sample, the simulator records the post-mutation quantile vector as the regression target. Although our problem statement is expressed in terms of future selectivity Q-Error, we train on corrected quantiles because, on the fixed histogram grid, those quantiles fully determine the single-column CDF consumed by the optimizer. Quantile regression is therefore a tractable surrogate for improving downstream selectivity and Q-Error while keeping the learning target engine-agnostic. The implementation is ~550 lines of pure Python + NumPy with ~38 K parameters, and the same shipped checkpoint generalizes across integer, floating-point, and date columns without per-installation retraining. Detailed layer sizes, attention-head configuration, optimization hyperparameters, and additional implementation notes are provided in the supplementary materials (`appendix/system_details.tex`).

3.4 Feedback Collection Architecture

A core practitioner requirement guides the feedback collection design: it must be implementable as a non-breaking addition to a production engine, without EXPLAIN ANALYZE, user intervention, or disruption to existing query traffic. We therefore use a two-layer design that captures per-conjunct selectivities while keeping the integration surface small.

Layer 1: Operator instrumentation. Filter-like operators maintain per-conjunct counters so that each conjunct in a compound predicate is accounted for independently while the final conjunction semantics remain unchanged. Crucially, this bookkeeping is piggybacked on the same tuple/page reads the engine already performs to evaluate the filter: no extra scan of the table file is introduced. The instrumentation therefore adds only a small number of counter updates during normal predicate evaluation. Detailed pseudocode for the instrumentation path is provided in the supplementary materials (`appendix/system_details.tex`).

Layer 2: Feedback listener plugin. Upon query completion, a listener plugin resolves each filter node’s predicate, table, and column identifiers, combines the per-conjunct counters into observed selectivities, and flushes structured observation tuples to disk for the Feature Tensorizer (§3.2). Because one tuple is emitted per conjunct, the write volume is bounded by the number of conditions in the query predicate; in particular, each query contributes at most as many observation records as there are filter conjuncts. This design confines instrumentation to a small number of execution operators and requires no CBO or planner modifications.

3.5 Non-Invasive Integration

OASIS integrates through two engine-facing shims: (1) a query-completion listener that reads existing runtime row-count metrics and emits observation tuples, and (2) a statistics-assembly hook that intercepts histogram reads immediately before they reach the CBO and substitutes corrected quantile values. Because correction is injected at the statistics assembly layer, no planner or cost-model changes are required. The correction backend can be embedded or remote, and the feature can be toggled per session for A/B testing or rollback.

3.6 Portable Statistics-Format Conversion Interfaces

A practical barrier to making statistics correction broadly useful is that many database systems do not expose a single self-contained histogram that can be edited in isolation. Instead, the stored statistics object often couples the histogram with auxiliary summaries whose probability mass must remain consistent after any update. PostgreSQL and Oracle use a *Most Common Values* (MCV) list plus a residual histogram; SQL Server combines a histogram with a density vector; and other engines expose yet other variants.

To make OASIS broadly deployable, we introduce a **statistics-format conversion interface**: a lightweight translation layer that maps engine-specific statistics layouts into a canonical full-distribution / quantile view, applies OASIS there, and translates the corrected result back. PostgreSQL is the representative stress case because its statistics split probability mass between an explicit MCV list and a residual histogram. Accordingly, the interface performs three phases: reconstruction of a canonical full distribution from the engine-native object, standard OASIS correction on that canonical view, and decomposition back to the native format. This paper’s end-to-end validation spans both deployment regimes targeted in this paper: PostgreSQL as the harder tightly coupled case, and MySQL as the self-contained histogram case. For self-contained histogram formats, the same correction path degenerates to direct histogram read-modify-write without coupled-state decomposition.

In the PostgreSQL case, explicit MCV entries become singleton masses and the residual histogram becomes equal-mass residual intervals in the canonical view. After correction, high-frequency singleton masses are promoted back into the MCV list and the remaining mass is resampled as residual histogram bounds. For self-contained formats such as MySQL and MariaDB, conversion degenerates to direct pass-through. For tightly coupled formats such as PostgreSQL and Oracle, it reconstructs and decomposes MCV-aware statistics objects. For hybrid formats such as SQL Server, it rebuilds the canonical view from histogram steps and density summaries, applies OASIS, then recomputes the auxiliary summaries.

This portability layer is validated in §4.2: on PostgreSQL-like defaults it adds only 0.066–0.073 ms overhead while preserving mass conservation and machine-precision round-trip fidelity. Formal reconstruction/decomposition algorithms, the detailed PostgreSQL MCV + histogram worked example, and extensions to other engines are provided in the supplementary materials (`appendix/mcv_adapter.tex`).

4 Experimental Evaluation

We organize the evaluation around four questions: **(1)** does the statistics-format conversion interface preserve correctness and low overhead; **(2)** how much does OASIS reduce histogram error as drift intensifies; **(3)** do those gains also appear in structural metrics, ablations, and initial-distribution generalization; and **(4)** do they translate into end-to-end optimizer gains across PostgreSQL and MySQL?

4.1 Experimental Setup

Unless otherwise stated, OASIS is trained once on synthetic Gaussian compound drift and reused without per-table retuning. All experiments run on a workstation with 32-core CPU, 128 GB RAM, Python 3.11; the correction model is pure Python with no ML framework dependencies.

Drift patterns. We evaluate OASIS primarily on Gaussian compound drift, where each round applies insert, delete, update, and null-change mutations; 6000 samples over $q \in \{1, 3, 5, 10, 15, 20\}$ are used for training, and $q \in \{1, 3, 5, 10, 15, 20, 25, 30\}$ for testing. Each sample contains a prior histogram after **ANALYZE**, 8–24 query observations collected during mutations, and a ground-truth corrected quantile vector. We separately test generalization across different initial distributions at fixed $q=10$ to isolate robustness beyond the Gaussian-mixture starting states seen during training.

Baselines and metrics. We compare OASIS against the uncorrected **Stale Prior** and three feedback-driven baselines consuming the same stale histogram and the same query-feedback source: **STHoles** [4], **ISOMER** [16], and **QuickSel-H** (our QuickSel-inspired histogram heuristic based on [18]). Unless otherwise stated, all feedback-driven methods use the same most-recent observation window $K=16$ and emit corrected quantiles on the same target histogram resolution, so the learned and classical updaters operate under a matched feedback budget and output interface. Baseline hyperparameters are fixed across drift levels and initial distributions; we do not retune individual methods per dataset or per drift intensity. For STHoles we use a hierarchical hole-tree instantiation adapted to the single-column setting: the original multidimensional holes degenerate to nested intervals, but updates still follow hole refinement rather than a flat split/merge smoother. For ISOMER we use cyclic relative-entropy projection on the exact interval partition induced by the active feedback predicates, dropping older constraints first when sequential drift makes the active set inconsistent. QuickSel, by contrast, is originally a selectivity learner rather than a histogram updater, so we evaluate a clearly labeled **QuickSel-H** adaptation that fits a feedback-constrained mixture-model CDF and then extracts quantiles from that CDF; we do not claim it is a line-by-line reproduction of the original QuickSel system. This matched-interface evaluation is the fairest comparison available for our problem setting because every method receives the same prior histogram, the

same bounded feedback window, and the same output obligation—namely, to return a corrected single-column histogram consumable by the CBO—without per-dataset retuning. Metrics include Q-Error (Eq. 1), Selectivity MAE, and Quantile MAE, averaged over 50 random test predicates per configuration. For the end-to-end benchmark we report wall-clock execution time on a TPC-DS workload (Scale Factor 10) under synchronized SCD Type 2 dimension growth on `item` and `customer` together with `store_sales` fact-table growth. We inject 10 drift rounds of roughly 3% each after the last statistics refresh; the PostgreSQL and MySQL protocols, together with the rationale for using execution time rather than aggregate Q-Error, are deferred to §4.6.

4.2 Statistics-Format Interface Validation

Before evaluating correction quality, we validate the PostgreSQL instantiation of the statistics-format conversion interface (§3.6). Because PostgreSQL tightly couples MCVs and histograms, it serves as the paper’s strongest portability stress test.

We generate synthetic columns (uniform, normal, Zipfian, bimodal) and construct PostgreSQL-style statistics with 10–100 MCV entries and 10–50 residual bins. We test exact round-trip translation between PostgreSQL format and OASIS’s canonical full-distribution view, then add a simulated correction step to isolate the effect of the MCV threshold τ_{MCV} . We measure round-trip accuracy, selectivity consistency, overhead, and threshold sensitivity.

Figure 1(a) shows that the interface preserves total probability mass exactly within floating-point precision: total mass error is 0 in all 20 round-trip trials, and residual-quantile MAE remains at machine precision (mean 2.4×10^{-17} , max 9.2×10^{-17}). Thus, reconstructing PostgreSQL’s compressed histogram as implicit equal-mass residual bins introduces no measurable drift in the threshold-free path.

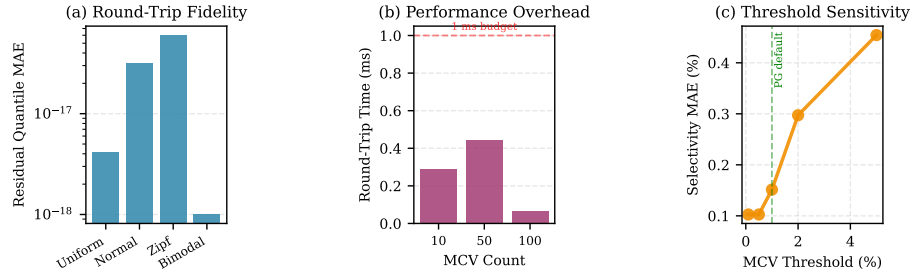


Fig. 1: Statistics-format interface validation on PostgreSQL-style MCV statistics. (a) Round-trip residual-quantile error stays at machine precision across all tested distributions. (b) Conversion overhead remains far below the planning-time budget in (D2) on typical PostgreSQL configurations. (c) Threshold sensitivity is mild near PostgreSQL’s default setting (1%) and increases gradually only at larger thresholds.

We also verify that PostgreSQL-style selectivity estimates remain consistent before and after round-trip conversion. For 100 random predicates per distribution (a mix of equality, range, and BETWEEN predicates), the global MAE is 1.1×10^{-4} ; the 95th percentile error is 10^{-4} , the 99th percentile is 3.8×10^{-3} , and the worst observed error remains below 0.39%. The small tail is confined to Zipfian cases with many ties, indicating negligible systematic bias.

Figure 1(b) reports conversion latency for realistic configurations. With PostgreSQL-like defaults (100 MCV entries, 10–20 residual bins), the full round-trip completes in just 0.066–0.073 ms. Reconstruction (Phase 1) remains below 0.03 ms across all tested settings, and even the largest stressed configuration (50 MCV entries, 50 residual bins) stays close to the 1 ms budget at 1.03 ms. The conversion interface therefore adds negligible overhead.

Finally, Figure 1(c) evaluates robustness to the MCV frequency threshold τ_{MCV} , which controls which singleton masses are promoted back into the PostgreSQL MCV list after correction. For thresholds in the range [0.1%, 1%], selectivity MAE stays low (0.10%–0.15%) while the MCV list remains nearly unchanged (20 to 17 entries). At higher thresholds (2%–5%), the MCV list shrinks more aggressively (9 to 3 entries), and MAE rises gradually to 0.45% at $\tau=5\%$ (99th percentile 1.95%). PostgreSQL’s default threshold ($\approx 1\%$) therefore provides a robust accuracy/compactness trade-off.

Overall, the PostgreSQL conversion layer preserves mass, preserves quantiles to machine precision, remains sub-millisecond in typical settings, and degrades gracefully as the MCV threshold rises. This validates the interface for DBMSs whose histograms are tightly coupled to auxiliary summaries.

4.3 Main Results: Q-Error vs. Drift Intensity

Table 1: Q-Error comparison across drift intensities ($\downarrow$ better). OASIS trained on $q \in \{1, 3, 5, 10, 15, 20\}$ (6000 samples total). Bold = best per row; “+ %” = improvement over the stale prior.

q	Stale Prior		STHoles		QuickSel-H		ISOMER		OASIS (ours)	
	Q-Err	—	Q-Err	+%	Q-Err	+%	Q-Err	+%	Q-Err	+%
1	1.188	—	1.089	+8.3%	1.193	-0.5%	1.092	+8.0%	1.147	+3.4%
3	1.460	—	1.175	+19.5%	1.331	+8.9%	1.184	+18.9%	1.185	+18.9%
5	1.749	—	1.246	+28.8%	1.478	+15.5%	1.247	+28.7%	1.213	+30.7%
10	2.633	—	1.568	+40.5%	1.996	+24.2%	1.544	+41.3%	1.267	+51.9%
15	2.995	—	1.590	+46.9%	2.053	+31.5%	1.649	+45.0%	1.338	+55.3%
20	3.028	—	1.548	+48.9%	2.048	+32.4%	1.668	+44.9%	1.262	+58.3%
25	3.383	—	1.647	+51.3%	2.216	+34.5%	1.733	+48.8%	1.285	+62.0%
30	3.220	—	1.499	+53.5%	1.962	+39.1%	1.716	+46.7%	1.255	+61.0%

Table 1 and Figure 2 evaluate OASIS on Gaussian compound drift, including unseen intensities $q \in \{25, 30\}$. The Stale Prior rises from 1.19 at $q=1$ to above 3.0 once $q \geq 15$, peaking at 3.38 on the unseen $q=25$ setting.

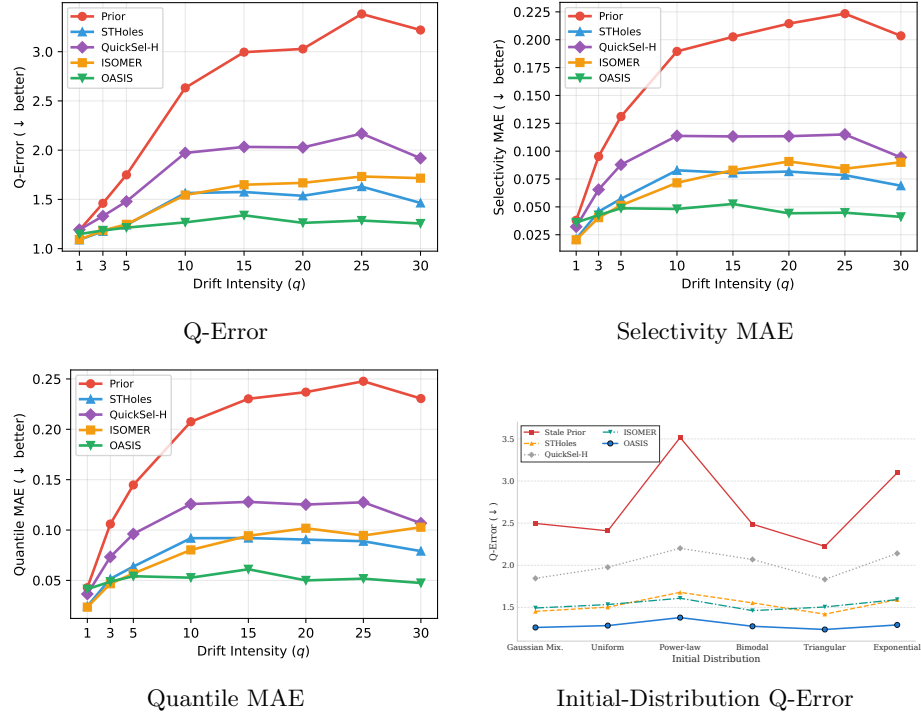


Fig. 2: Correction performance across drift intensities and initial distributions. The first three panels report Gaussian compound-drift results for $K=16$ and $B=10$; the fourth panel summarizes Q-Error under unseen initial distributions at $q=10$. OASIS achieves up to 62.0% Q-Error reduction over the stale prior while remaining robust across different starting distributions.

In-distribution performance and unseen intensities. At $q=1$, the hierarchical STHoles baseline is strongest, indicating that very mild drift can already be captured by localized hole refinement while OASIS remains deliberately conservative under desideratum **D3**. At $q=3$, OASIS nearly ties the strongest classical refiners but STHoles still remains best. From $q \geq 5$, OASIS is the best method at every drift level under the same $K=16$ feedback budget, and the gap widens as the histogram becomes more stale. At $q=10$ – 30 , OASIS reduces Q-Error by 51.9%–62.0% versus the Stale Prior, and this advantage persists on the unseen test intensities $q=25$ and 30 . The strongest classical competitor alternates between hierarchical STHoles and ISOMER, but neither matches OASIS once drift is no longer near-stale.

4.4 Generalization Across Initial Distributions

To move beyond a single drift family, we also vary the *initial* table shape before drift begins. Applying the same mutation process to different starting histograms yields different post-drift distributions, providing a direct test of whether the

Table 2: Q-Error across different initial distributions at $q=10$. A model trained only on Gaussian-mixture initial states remains best on all tested distributions.

Initial Distribution	Stale	STHoles	QuickSel-H	ISOMER	OASIS
Gaussian Mixture	2.497	1.453	1.845	1.493	1.262
Uniform	2.409	1.503	1.977	1.534	1.284
Skewed (Power-law)	3.517	1.679	2.202	1.609	1.379
Bimodal	2.488	1.554	2.069	1.462	1.276
Triangular	2.225	1.420	1.833	1.505	1.238
Exponential	3.097	1.592	2.141	1.593	1.292

model generalizes beyond one specific distribution family. This is the right stress case for OASIS because, at inference time, the model still sees only the stale prior histogram and the most recent query-feedback window, rather than any explicit label describing the source distribution.

We therefore vary the initial table shape to test whether generalization depends on the pre-drift distribution used during training:

- **Uniform**: Equal probability across the value range.
- **Skewed (Power-law)**: 80-20 distribution simulating real-world activity.
- **Bimodal**: Two distinct peaks representing distinct user segments.
- **Triangular**: Peak at center, tapering to edges.
- **Exponential**: Decay from low to high values.
- **Gaussian Mixture**: Original training distribution (baseline).

Table 2 shows consistent 44.4%–60.8% Q-Error reduction across all tested initial distributions, even though training uses only Gaussian-mixture initial states. At $q=10$, OASIS remains best on every evaluated shape, with Q-Error ranging from 1.238 to 1.379 versus 2.225 to 3.517 for the stale prior. The strongest classical baseline alternates between hierarchical STHoles and ISOMER depending on the distribution, but neither matches OASIS on any tested initial shape. This supports the claim that the normalized quantile representation transfers beyond the exact initial shape seen during training rather than overfitting to the Gaussian-mixture start state.

4.5 Structural Accuracy: Quantile and Selectivity Error

Beyond cross-distribution Q-Error robustness, we also examine whether OASIS improves the underlying histogram shape itself. Table 3 complements Q-Error with two structural metrics: Quantile MAE for boundary recovery and Selectivity MAE for CDF quality. OASIS is the best method on both metrics for every drift level $q \geq 5$. At $q=10$, Quantile MAE drops from 0.208 to **0.053** and Selectivity MAE from 0.190 to **0.048**, substantially below both ISOMER (0.080 / 0.072) and hierarchical STHoles (0.093 / 0.083). The joint gain in both metrics indicates that OASIS reconstructs histogram shape rather than merely matching observed

predicates. At $q=1$ and $q=3$, ISOMER marginally leads because the prior is still close to the true distribution and only limited correction is needed.

Table 3: Quantile MAE and Selectivity MAE across drift intensities (training: $q \in \{1, 3, 5, 10, 15, 20\}$, $k_{\text{train}}=1000$ per level). Bold denotes best method per metric.

q	Quantile MAE ($\downarrow$)					Selectivity MAE ($\downarrow$)				
	Prior	STHoles	QSel-H	ISOMER	OASIS	Prior	STHoles	QSel-H	ISOMER	OASIS
1	0.043	0.025	0.036	0.023	0.041	0.038	0.021	0.031	0.020	0.036
3	0.106	0.052	0.071	0.047	0.049	0.095	0.046	0.064	0.040	0.043
5	0.145	0.065	0.094	0.057	0.054	0.131	0.059	0.085	0.051	0.049
10	0.208	0.093	0.126	0.080	0.053	0.190	0.083	0.114	0.072	0.048
15	0.230	0.095	0.128	0.094	0.061	0.203	0.082	0.113	0.083	0.053
20	0.237	0.092	0.127	0.102	0.050	0.214	0.083	0.115	0.091	0.044
25	0.248	0.092	0.131	0.095	0.052	0.223	0.081	0.118	0.084	0.045
30	0.231	0.083	0.111	0.103	0.047	0.204	0.072	0.098	0.090	0.041

4.6 End-to-End Evaluation on TPC-DS

This subsection asks whether the single-column gains of OASIS translate into system-level optimizer benefits on PostgreSQL 14, which uses the full conversion interface. Because aggregate workload Q-Error would mix histogram quality with other planning factors, we use execution time as the end-to-end metric.

PostgreSQL: tightly coupled statistics under schema-level drift. We evaluate on TPC-DS at Scale Factor 10 using an analytical SQL workload on PostgreSQL 14. Starting from fresh statistics, we inject drift for 10 rounds of roughly 3% each. The drift applies SCD Type 2 dimension growth to the two large dimension tables `item` and `customer`: in each round we expire a subset of currently valid rows and insert successor rows with new surrogate keys. We then insert new `store_sales` fact rows whose join predicates reference these newly introduced out-of-bounds keys. This changes the support of filtered attributes while preserving realistic schema-level relationships more faithfully than synthetic single-column drift.

We compare three strategies: leaving statistics stale, applying OASIS, and running Full ANALYZE. PostgreSQL’s cost-based optimizer uses equi-depth histograms for range-predicate selectivity and NDV for equality predicates, so corrected histogram statistics propagate directly to range-filter selectivity estimates that influence join ordering on drifted columns.

OASIS reduces total runtime by 24.9 s relative to the stale baseline, while Full ANALYZE reduces it by 47.9 s. Thus, OASIS recovers $24.9/47.9 = 52.0\%$ of the end-to-end improvement delivered by Full ANALYZE. This is notable because the correction targets only the histogram component of the statistics object: base-count statistics remain stale even under continued fact-table growth, yet a large fraction of the optimizer benefit is still recovered.

Table 4: TPC-DS workload execution time (PostgreSQL 14, Scale Factor 10, 10 drift rounds at $\approx 3\%$ each). OASIS recovers 52.0% of Full ANALYZE’s improvement over stale statistics.

Method	Total time (s)	vs. Stale
Stale Prior	364.2	—
OASIS	339.3	−6.8%
Full ANALYZE	316.3	−13.2%

Table 5: Additional TPC-DS random-offset stress case on PostgreSQL 14. Here Full ANALYZE is 11.1% slower than the stale baseline, while OASIS remains 2.7% faster.

Method	Total time (s)	vs. Stale
Stale Prior	181.2	—
OASIS	176.3	−2.7%
Full ANALYZE	201.4	+11.1%

Additional random-offset stress case: Full ANALYZE can be non-monotonic. To illustrate more directly that end-to-end runtime depends on optimizer internals rather than on statistics freshness alone, we also report an additional TPC-DS case obtained after repeated randomized offset drift injections, selecting a representative run in which Full ANALYZE performs worse than leaving the statistics stale. The DBMS and workload remain the same; only the realized drift pattern differs.

In this additional case, OASIS still improves over the stale baseline by 4.9 s, whereas Full ANALYZE slows the workload by 20.2 s. We report this result as a diagnostic counterexample, not as a replacement for the main average trend above. The point is not that fresher statistics are generally harmful; rather, end-to-end runtime is not monotone in statistics freshness once changed estimates cross optimizer decision boundaries and trigger different plans. A plausible interpretation is that targeted single-column correction can sometimes remain on the beneficial side of that boundary even when a full statistics refresh does not. This case therefore reinforces our central scope claim: OASIS directly improves stale single-column optimizer inputs, while the final planning outcome remains DBMS- and optimizer-dependent.

MySQL: self-contained histogram deployment. We next validate the lightweight deployment regime on MySQL 8.0.31+, whose column histograms are self-contained rather than coupled to an MCV-style residual object. In this setting, OASIS is realized through the engine’s standalone column-histogram path; unlike PostgreSQL, no MCV-aware reconstruction/decomposition layer is required. For consistency with the rest of the paper, we report this MySQL intervention simply as OASIS.

We compare leaving statistics stale, applying OASIS, and running Full ANALYZE. In MySQL, the OASIS path operates directly on the self-contained

Table 6: TPC-DS workload execution time on MySQL 8.0.31+ under the self-contained histogram setting. This experiment validates the lightweight OASIS deployment path, where histogram correction does not require coupled-state reconstruction.

Method	Total time (s)	vs. Stale
Stale Prior	710.8	—
OASIS	623.4	−12.3%
Full ANALYZE	595.4	−16.2%

histogram object, making it the lightweight counterpart to the PostgreSQL deployment above.

On MySQL, OASIS reduces total runtime by 87.4 s relative to the stale baseline, while Full ANALYZE reduces it by 115.4 s. Thus, OASIS recovers $87.4/115.4 = 75.7\%$ of the end-to-end improvement delivered by Full ANALYZE. Unlike the PostgreSQL experiment, this result does not require coupled-state translation: OASIS acts directly on the standalone histogram representation that the optimizer consumes. Together with the PostgreSQL results above, the MySQL result shows that OASIS spans both deployment regimes targeted in this paper: tightly coupled statistics objects and self-contained histograms.

4.7 OASIS Overhead

Because §4.2 already establishes that the statistics-format conversion interface adds only sub-millisecond overhead, we isolate the runtime of the learned correction pipeline itself here. In a single-histogram microbenchmark on the same workstation, the deployed OASIS v2 implementation averages 1.06 ms per correction event over 1000 trials, including dynamic tensorization of the most recent $K=16$ observations and one forward pass of the 38K-parameter MLP. This runtime remains well below the planning-time budget in (D2), indicating that the learned correction pipeline adds only a small per-column overhead in online deployment.

5 Related Work

Classical histogram methods construct statistics from data scans [8,19], while stream summaries [9,2] provide provable accuracy bounds. Feedback-driven variants like STHoles [4], self-tuning histograms [1], and ISOMER [16] refine boundaries using execution feedback, but typically suffer from slow convergence or require table-specific parameter tuning. QuickSel [18] uses mixture models for points estimates rather than full CDF correction. In contrast, OASIS trains on diverse synthetic drift scenarios, enabling zero-shot deployment without per-table calibration.

Machine learning for query optimization spans diverse approaches, including learned cost models and indexes [12,22,23]. Learned cardinality estimators (e.g.,

NeuroCard [24], FACE [25], DeepDB [7], MSCN [11], Naru [26]) predict join sizes but output specific point estimates rather than functional histograms. A particularly close neighboring direction is copula-based learned cardinality estimation, e.g., CoLSE [20], which uses copula theory to model joint probabilities for single-table queries. By construction, such models combine one-dimensional marginals with a learned dependence structure, so accurate single-column statistics remain a prerequisite rather than a substitute. As recent benchmarks demonstrate [5,6], learned estimators often entail substantial training cost or additional modeling complexity. Meanwhile, plan-level optimizers (e.g., Bao [14], Neo [15], LEO [21], Eddies [3], and mid-query re-optimization [10]) use execution history to override plan choices entirely. These methods fundamentally differ from OASIS, requiring deep CBO modifications. OASIS operates transparently at the statistics level, generating corrected histograms that plug directly into existing CBOs. These approaches are complementary; future work could combine them, using OASIS to provide robust base-table inputs [13] for learned cross-column estimators.

6 Conclusion

We presented OASIS, a lightweight, feedback-driven framework that repairs stale column statistics online from query execution observations, without table scans or manual tuning. By formulating correction as a regression problem and training an attention-pooled MLP once on diverse synthetic drift profiles, OASIS ships as a single 38K-parameter checkpoint that deploys zero-shot across arbitrary schemas—no per-table optimizer surgery and no schema-specific configuration. Integration requires only two non-invasive engine hooks (a query-completion listener and a statistics-assembly intercept), leaving the CBO and execution engine unchanged. A key enabler of broad deployment is the statistics-format conversion interface, which extends correction beyond engines with standalone histograms to systems where histogram state is tightly coupled with MCV lists or density summaries. Experiments confirm up to 62.0% Q-Error reduction on compound drift, 44.4%–60.8% Q-Error reduction across unseen initial distributions, and recovery of 52.0% and 75.7% of Full ANALYZE’s end-to-end execution-time improvement on PostgreSQL 14 and MySQL 8.0.31+ TPC-DS, respectively, through OASIS. These results demonstrate that implicit query feedback is a practical and nearly zero-additional-I/O signal for sustaining optimizer accuracy against continuous data drift: observations are collected during normal predicate evaluation, require no extra table scan, and write only a bounded number of records per query. More precisely, OASIS improves the accuracy of stale single-column statistics; the extent to which this yields better plans depends on how a particular DBMS optimizer consumes those statistics internally. OASIS is intentionally scoped to marginal single-column repair rather than joint-distribution modeling, making it complementary to methods that target multi-column correlations, extended statistics, or join-level estimation errors. Given its lightweight runtime cost and non-invasive integration footprint, OASIS represents a compelling addition to

production database systems seeking to maintain optimizer quality between scheduled ANALYZE passes.

References

1. Aboulnaga, A., Chaudhuri, S.: Self-tuning histograms: Building histograms without looking at data. In: ACM SIGMOD Record. vol. 28, pp. 181–192 (1999)
2. Agarwal, P.K., Cormode, G., Huang, Z., Phillips, J.M., Wei, Z., Yi, K.: Mergeable summaries. In: ACM Transactions on Database Systems. vol. 38, pp. 1–28 (2013)
3. Avnur, R., Hellerstein, J.M.: Eddies: Continuously adaptive query processing. In: ACM SIGMOD Record. vol. 29, pp. 261–272 (2000)
4. Bruno, N., Chaudhuri, S., Gravano, L.: STHoles: A multidimensional workload-aware histogram. In: Proceedings of SIGMOD. pp. 211–222 (2001)
5. Dutt, A., Wang, C., Nazi, A., Kandula, S., Narasayya, V., Chaudhuri, S.: Are we ready for learned cardinality estimation? In: Proceedings of the VLDB Endowment. vol. 14, pp. 1640–1654 (2021)
6. Han, Y., Wu, Z., Wu, P., Zhu, R., Yang, J., Tan, L.W., Zeng, K., Cai, G., Zhang, Y., Li, G.: Cardinality estimation in DBMS: A comprehensive benchmark evaluation. In: Proceedings of the VLDB Endowment. vol. 15, pp. 752–765 (2021)
7. Hilprecht, B., Schmidt, A., Kulesa, M., Molina, A., Kersting, K., Binnig, C.: DeepDB: Learn from data, not from queries! In: Proceedings of the VLDB Endowment. vol. 13, pp. 992–1005 (2020)
8. Ioannidis, Y.E., Poosala, V.: Universality of serial histograms. In: Proceedings of the 19th VLDB Conference. pp. 256–267 (1993)
9. Ivkin, N., Nelson, J., Yu, H., Braverman, V.: KLL: Approximately optimal streaming quantile estimation. Proceedings of the ACM on Management of Data **2**(1), 1–23 (2019)
10. Kabra, N., DeWitt, D.J.: Efficient mid-query re-optimization of sub-optimal query execution plans. In: ACM SIGMOD Record. vol. 27, pp. 106–117 (1998)
11. Kipf, A., Kipf, T., Radke, B., Leis, V., Boncz, P., Kemper, A.: Learned cardinalities: Estimating correlated joins with deep learning. In: CIDR (2019)
12. Kraska, T., Beutel, A., Chi, E.H., Dean, J., Polyzotis, N.: The case for learned index structures. Proceedings of SIGMOD pp. 489–504 (2018)
13. Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., Neumann, T.: How good are query optimizers, really? In: Proceedings of the VLDB Endowment. vol. 9, pp. 204–215 (2015)
14. Marcus, R., Negi, P., Mao, H., Tatbul, N., Alizadeh, M., Kraska, T.: Bao: Making learned query optimization practical. In: Proceedings of SIGMOD. pp. 1275–1288 (2022)
15. Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, M., Kraska, T., Papaemmanouil, O., Tatbul, N.: Neo: A learned query optimizer. In: Proceedings of the VLDB Endowment. vol. 12, pp. 1705–1718 (2019)
16. Markl, V., Megiddo, N., Kutsch, M., Tran, T.M., Lang, P., Raman, V.: Consistent selectivity estimation via maximum entropy. The VLDB Journal **16**(2), 169–188 (2007)
17. Moerkotte, G., Neumann, T., Steidl, G.: Preventing bad plans by bounding the impact of cardinality estimation errors. In: Proceedings of the VLDB Endowment. vol. 2, pp. 982–993 (2009)

18. Park, Y., Zhong, S., Mozafari, B.: QuickSel: Quick selectivity learning with mixture models. In: *Proceedings of SIGMOD*. pp. 1017–1033 (2020)
19. Poosala, V., Ioannidis, Y.E.: Selectivity estimation without the attribute value independence assumption. In: *Proceedings of the 23rd VLDB Conference*. pp. 486–495 (1997)
20. Rathuwadu, L., Liu, G., Leckie, C., Borovica-Gajic, R.: Colse: A lightweight and robust hybrid learned model for single-table cardinality estimation using joint cdf. *arXiv preprint arXiv:2512.12624* (2025). <https://doi.org/10.48550/arXiv.2512.12624>
21. Stillger, M., Lohman, G., Markl, V., Kandil, M.: LEO – DB2’s learning optimizer. In: *Proceedings of the 27th VLDB Conference*. pp. 19–28 (2001)
22. Sun, J., Li, G.: An end-to-end learning-based cost estimator. In: *Proceedings of the VLDB Endowment*. vol. 13, pp. 307–319 (2019)
23. Wu, W., Chi, Y., Zhu, H., Tatemura, J., Hacigümüş, H., Naughton, J.F.: Towards a learning optimizer for shared clouds. In: *Proceedings of the VLDB Endowment*. vol. 12, pp. 210–222 (2018)
24. Yang, Z., Kamsetty, A., Luan, S., Liang, E., Duan, Y., Chen, X., Stoica, I.: NeuroCard: One cardinality estimator for all tables. In: *Proceedings of the VLDB Endowment*. vol. 14, pp. 61–73 (2020)
25. Yang, Z., Liang, E., Duan, Y., Luan, S., Stoica, I., Hellerstein, J.M.: FACE: A normalizing flow based cardinality estimator. In: *SIGMOD Workshop on Approximate Query Processing* (2020)
26. Yang, Z., Liang, E., Kamsetty, A., Wu, C., Duan, Y., Chen, X., Abbeel, P., Hellerstein, J.M., Krishnan, S., Stoica, I.: Deep unsupervised cardinality estimation. *Proceedings of the VLDB Endowment* **13**(3), 279–292 (2019)